

LAPORAN PRATIKUM
STRUKTUR DATA
IMPLEMENTASI STRUKTUR DATA LINKEDLIST

Diususun Oleh :

Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Dr. Wahyudi, S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa berkat rahmat dan karunianya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 5 dengan judul “Implementasi Struktur Data LinkedList” ini dengan baik. Pada laporan praktikum pekan satu ini akan membahas mengenai penerapan konsep ADT (Abstract Data Type) spesifiknya membahas Struktur data LinkedList.

Penulis menyadari bahwa laporan ini tidak terlepas dari kesalahan dan kekurangan baik dari penulisan dan pembahasan. Oleh karena itu penulis ingin mengucapkan terimakasih kepada dosen pengampu Bapak Dr. Wahyudi, S.T M.T. dan asisten praktikum Muhammad Galid Averro yang telah memberikan arahan selama proses praktikum dan proses penyusunan laporan praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dalam memahami struktur data LinkedList dan penerapannya dalam java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.	1
1.2 Tujuan Praktikum.	1
1.3 Manfaat praktikum.	1
BAB II PEMBAHASAN	2
2.1 NodeSLL	2
2.1.1 Penjelasan Program.....	2
2.1.2 Kode Program	2
2.2 Tambah SLL	3
2.2.1 Penjelasan Program.....	3
2.2.2 Kode Program dan Output	3
2.3 Hapus SLL	5
2.3.1 Penjelasan Program.....	5
2.3.2 Kode Program dan Output	5
2.4 Pencarian SLL	7
2.4.1 Penjelasan Program.....	7
2.4.2 Kode Program dan Output	7
BAB III PENUTUP	9
DAFTAR PUSTAKA	10
TUGAS PRAKTIKUM.....	11

BAB I

PENDAHULUAN

1.1 Latar Belakang.

Dalam pengembangan perangkat lunak, efisiensi pengelolaan memori menjadi aspek krusial, terutama ketika menangani sekumpulan data yang jumlahnya bersifat dinamis. Struktur data statis seperti array sering kali menghadapi kendala keterbatasan ukuran yang tetap dan biaya komputasi yang tinggi saat melakukan operasi penyisipan atau penghapusan elemen di posisi tertentu. LinkedList mampu mengalokasikan memori secara dinamis menggunakan konsep node yang saling terhubung melalui referensi atau pointer. Dengan memanfaatkan hal ini dalam bahasa pemrograman Java, pengolahan data dapat dilakukan secara lebih fleksibel tanpa perlu melakukan pergeseran elemen secara manual di dalam memori.

1.2 Tujuan Praktikum.

- Memahami konsep dasar, arsitektur node, serta mekanisme kerja struktur data LinkedList dalam manajemen memori dinamis.
- Mampu mengimplementasikan operasi-operasi fundamental seperti penambahan (insert), penghapusan (delete), dan pencarian (search) elemen pada LinkedList dengan bahasa pemrograman Java.

1.3 Manfaat praktikum.

Praktikum ini memberikan manfaat bagi mahasiswa dalam mengasah logika pemrograman, khususnya pada pengelolaan referensi objek dan struktur data non statis. Dengan menguasai LinkedList, selain itu, pemahaman ini memungkinkan mahasiswa untuk memilih struktur data yang paling optimal dalam menyelesaikan permasalahan pemrograman, sehingga aplikasi yang dihasilkan memiliki performa dan manajemen sumber daya yang lebih efisien.

BAB II PEMBAHASAN

2.1 NodeSLL

2.1.1 Penjelasan Program

Program mendefinisikan blueprint dari sebuah simpul dalam senarai berantai. Kelas NodeSLL_2511532008 merepresentasikan satu unit data yang memiliki dua komponen utama: variabel data_2008 untuk menyimpan nilai integer dan objek next_2008 yang berfungsi sebagai pointer atau referensi untuk menghubungkan simpul tersebut ke simpul berikutnya dalam memori. Melalui konstruktor NodeSLL_2511532008, setiap simpul baru yang dibuat akan menginisialisasi nilainya sendiri dan mengatur referensi lanjutannya ke null, menandakan bahwa simpul tersebut belum terhubung ke elemen lain pada saat instansiasi awal.

2.1.2 Kode Program

```
1 package pekan5_2511532008;
2
3 public class NodeSLL_2511532008 {
4
5     int data_2008;
6     NodeSLL_2511532008 next_2008; // --->> Pointer
7
8     public NodeSLL_2511532008(int data_2008) {
9         this.data_2008 = data_2008;
10        this.next_2008 = null;
11    }
12 }
13
```

Gambar 2.1

2.2 Tambah SLL

2.2.1 Penjelasan Program

Program yang mengimplementasikan penambahan data pada tiga kondisi berbeda. Metode `insertAtFront` memungkinkan penambahan data di posisi paling depan dengan mengubah pointer simpul baru agar mengarah ke `head_2008` lama, lalu menjadikannya sebagai head baru. Metode `insertAtEnd` melakukan penelusuran (traversal) terlebih dahulu hingga menemukan ujung senarai sebelum menyisipkan simpul `newNode_2008`. Sementara itu, metode `insertPos_2008` memiliki logika yang lebih kompleks karena harus menelusuri senarai hingga mencapai posisi spesifik yang diinginkan oleh pengguna.

2.2.2 Kode Program dan Output

```
public class TambahSLL_2511532008 {  
    public static NodeSLL_2511532008 insertAtFront(NodeSLL_2511532008 head_2008, int value_2008) {  
        NodeSLL_2511532008 new_node_2008 = new NodeSLL_2511532008(value_2008);  
        new_node_2008.next_2008 = head_2008;  
        return new_node_2008;  
    }  
    public static NodeSLL_2511532008 insertAtEnd(NodeSLL_2511532008 head_2008, int value_2008) {  
        NodeSLL_2511532008 new_node_2008 = new NodeSLL_2511532008(value_2008);  
        if (head_2008 == null) {  
            return new_node_2008;  
        }  
        NodeSLL_2511532008 last_2008 = head_2008;  
        while (last_2008.next_2008 != null) {  
            last_2008 = last_2008.next_2008;  
        }  
        last_2008.next_2008 = new_node_2008;  
        return head_2008;  
    }  
    static NodeSLL_2511532008 GetNode_2008 (int data_2008) {  
        return new NodeSLL_2511532008(data_2008);  
    }  
    static NodeSLL_2511532008 insertPos_2008(NodeSLL_2511532008 headNode_2008, int position_2008, int value_2008) {  
        NodeSLL_2511532008 head_2008 = headNode_2008;  
        if (position_2008 < 1)  
            System.out.print("Invalid position");  
        if (position_2008 == 1) {  
            NodeSLL_2511532008 new_node_2008 = new NodeSLL_2511532008(value_2008);  
            new_node_2008.next_2008 = head_2008;  
            return new_node_2008;  
        } else {  
            while (position_2008 -- != 0) {  
                if (position_2008 == 1) {  
                    NodeSLL_2511532008 new_node_2008 = GetNode_2008(value_2008);  
                    new_node_2008.next_2008 = headNode_2008.next_2008;  
                    headNode_2008.next_2008 = new_node_2008;  
                    break;  
                }  
                headNode_2008 = headNode_2008.next_2008;  
            }  
        }  
        if (position_2008 != 1)  
            System.out.print("Posisi di luar jangkauan");  
        return head_2008;  
    }  
}
```

Gambar 2.2

```

public static void printList_2008(NodeSLL_2511532008 head_2008) {
    NodeSLL_2511532008 curr_2008 = head_2008;
    while (curr_2008.next_2008 != null) {
        System.out.print(curr_2008.data_2008+" --> ");
        curr_2008 = curr_2008.next_2008;
    }
    if (curr_2008.next_2008==null) {
        System.out.print(curr_2008.data_2008);
    }
    System.out.println();
}

public static void main(String[] args) {
    NodeSLL_2511532008 head_2008 = new NodeSLL_2511532008(2);
    head_2008.next_2008 = new NodeSLL_2511532008(3);
    head_2008.next_2008.next_2008 = new NodeSLL_2511532008(5);
    head_2008.next_2008.next_2008.next_2008 = new NodeSLL_2511532008(6);

    System.out.print("Senarai berantai awal: ");
    printList_2008(head_2008);

    System.out.print("tambah 1 simpul di depan: ");
    int data_2008 = 1;
    head_2008 = insertAtFront(head_2008, data_2008);

    printList_2008(head_2008);
    System.out.print("tambah 1 simpul di belakang: ");
    int data2_2008 = 7;
    head_2008 = insertAtEnd(head_2008, data2_2008);

    printList_2008(head_2008);

    System.out.print("Tambah 1 simpul data ke 4 : ");
    int data3_2008 = 4;
    int pos_2008 = 4;
    head_2008 = insertPos_2008 (head_2008, pos_2008, data3_2008);
    printList_2008(head_2008);
}

```

Gambar 2.3

```

Senarai berantai awal: 2 --> 3 --> 5 --> 6
tambah 1 simpul di depan: 1 --> 2 --> 3 --> 5 --> 6
tambah 1 simpul di belakang: 1 --> 2 --> 3 --> 5 --> 6 --> 7
Tambah 1 simpul data ke 4 : 1 --> 2 --> 3 --> 4 --> 5 --> 6 --> 7

```

Gambar 2.4

2.3 Hapus SLL

2.3.1 Penjelasan Program

Program penghapusan SLL. Metode deleteHead_2008 bekerja secara efisien dengan memutus hubungan pertama dan menggeser referensi head_2008 ke simpul kedua. Pada metode removeLastNode_2008, program mencari simpul terakhir untuk memutus hubungan next_2008 agar simpul terakhir terhapus dari struktur. Metode deleteNode_2008 memberikan fleksibilitas dengan menghapus elemen proses ini dilakukan dengan cara menghubungkan simpul sebelum target prev_2008 langsung ke simpul sesudah target, sehingga simpul pada posisi yang ditentukan terlompati dan tidak lagi menjadi bagian dari rantai data.

2.3.2 Kode Program dan Output

```
public class HapusSLL_2511532008 {
    public static NodeSLL_2511532008 deleteHead_2008(NodeSLL_2511532008 head_2008) {
        if (head_2008 == null)
            return null;
        head_2008 = head_2008.next_2008;
        return head_2008; }
    public static NodeSLL_2511532008 removeLastNode_2008(NodeSLL_2511532008 head_2008) {
        if (head_2008 == null) {
            return null;
        }
        if (head_2008.next_2008 == null) {
            return null;
        }
        NodeSLL_2511532008 secondLast_2008 = head_2008;
        while (secondLast_2008.next_2008.next_2008 != null) {
            secondLast_2008 = secondLast_2008.next_2008;
        }
        secondLast_2008.next_2008 = null;
        return head_2008; }
    public static NodeSLL_2511532008 deleteNode_2008(NodeSLL_2511532008 head_2008, int position_2008) {
        NodeSLL_2511532008 temp_2008 = head_2008;
        NodeSLL_2511532008 prev_2008 = null;
        if (temp_2008 == null)
            return head_2008;
        if (position_2008 == 1) {
            head_2008 = temp_2008.next_2008;
            return head_2008;
        }
        for (int i_2008 = 1; temp_2008 != null && i_2008 < position_2008; i_2008++) {
            prev_2008 = temp_2008;
            temp_2008 = temp_2008.next_2008;
        }
        if (temp_2008 != null) {
            prev_2008.next_2008 = temp_2008.next_2008;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_2008; }
    public static void printList_2008(NodeSLL_2511532008 head_2008) {
        NodeSLL_2511532008 curr_2008 = head_2008;
        while (curr_2008.next_2008 != null) {
            System.out.print(curr_2008.data_2008 + "-->");
            curr_2008 = curr_2008.next_2008;
        }
        if (curr_2008.next_2008 == null) {
            System.out.print(curr_2008.data_2008);
        }
        System.out.println();
    }
}
```

Gambar 2.5

```

    public static void main(String[] args) {
        NodeSLL_2511532008 head_2008 = new NodeSLL_2511532008(1);
        head_2008.next_2008 = new NodeSLL_2511532008(2);
        head_2008.next_2008.next_2008 = new NodeSLL_2511532008(3);
        head_2008.next_2008.next_2008.next_2008 = new NodeSLL_2511532008(4);
        head_2008.next_2008.next_2008.next_2008.next_2008 = new NodeSLL_2511532008(5);
        head_2008.next_2008.next_2008.next_2008.next_2008.next_2008 = new NodeSLL_2511532008(6);
        System.out.println("list awal: ");
        printList_2008(head_2008);

        head_2008 = deleteHead_2008(head_2008);
        System.out.println("List setelah head dihapus: ");
        printList_2008(head_2008);

        head_2008 = removeLastNode_2008(head_2008);
        System.out.println("List setelah simpul terakhir di hapus: ");
        printList_2008(head_2008);

        int position_2008 = 2;
        head_2008 = deleteNode_2008(head_2008, position_2008);

        System.out.println("List setelah posisi 2 dihapus: ");
        printList_2008(head_2008);
    }
}

```

Gambar 2.6

```

list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

Gambar 2.7

2.4 Pencarian SLL

2.4.1 Penjelasan Program

Program dengan fitur akses informasi. Metode `traversal_2008` berfungsi untuk mengunjungi setiap simpul satu per satu menggunakan perulangan `while` dan mencetak nilai `data_2008` ke layar hingga mencapai ujung senarai yang ditandai dengan nilai `null`. Selain penelusuran, terdapat metode `searchKey_2008` yang menerapkan algoritma linear search. Metode ini membandingkan kunci pencarian `key_2008` dengan data pada setiap simpul, jika ditemukan kecocokan, program akan mengembalikan nilai boolean `true`. Implementasi ini sangat penting untuk memverifikasi keberadaan suatu informasi di dalam koleksi data yang dinamis sebelum melakukan operasi lanjutan.

2.4.2 Kode Program dan Output

```
public class PencarianSLL_2511532008 {

    static boolean searchKey_2008 (NodeSLL_2511532008 head_2008, int key_2008) {
        NodeSLL_2511532008 curr_2008 = head_2008;
        while (curr_2008 != null) {
            if ( curr_2008.data_2008 == key_2008)
                return true;
            curr_2008 = curr_2008.next_2008;}
        return false;
    }

    public static void traversal_2008 (NodeSLL_2511532008 head_2008) {

        NodeSLL_2511532008 curr_2008 = head_2008;

        while (curr_2008 != null) {
            System.out.print(" " + curr_2008.data_2008);
            curr_2008 = curr_2008.next_2008;
        }
        System.out.println();
    }

    public static void main (String [] args) {
        NodeSLL_2511532008 head_2008 = new NodeSLL_2511532008(14);
        head_2008.next_2008 = new NodeSLL_2511532008(21);
        head_2008.next_2008.next_2008 = new NodeSLL_2511532008(13);
        head_2008.next_2008.next_2008.next_2008 = new NodeSLL_2511532008(30);
        head_2008.next_2008.next_2008.next_2008.next_2008 = new NodeSLL_2511532008(10);
        System.out.print("Penelusuran : ");
        traversal_2008 (head_2008);

        int key_2008 = 30;
        System.out.print("Cari data " + key_2008 + " = ");
        if (searchKey_2008(head_2008, key_2008))
            System.out.println("Ketemu");
        else
            System.out.println(" Tidak ada ");
    }
}
```

Gambar 2.8

```
Penelusuran : 14 21 13 30 10  
Cari data 30 = Ketemu
```

Gambar 2.9

BAB III

PENUTUP

Berdasarkan hasil praktikum, dapat disimpulkan bahwa struktur data Single Linked List (SLL) merupakan metode pengelolaan data linear yang sangat fleksibel karena memanfaatkan alokasi memori secara dinamis. Melalui penggunaan simpul (node) dan referensi antar objek (pointer), program dapat melakukan operasi penyisipan, penghapusan, dan pencarian elemen secara efisien tanpa bergantung pada ukuran memori yang statis seperti pada struktur array. Keberhasilan implementasi operasi-operasi tersebut dalam bahasa Java sangat bergantung pada ketelitian manipulasi penunjuk referensi untuk menjaga agar rantai data tetap terhubung dengan benar, sehingga menghasilkan sistem manajemen data yang adaptif dan optimal.

DAFTAR PUSTAKA

W3Schools, “Java Tutorial,” 2026. [Daring]. Tersedia pada:
<https://www.w3schools.com/java/default.asp>.

TUGAS PRAKTIKUM

Kelas ADT

```
package pekan5_2511532008;

public class Pasien_2511532008 {
    private String namaPasien_2008;
    private String keluhan_2008;
    private int nomorAntrian_2008;
    private Pasien_2511532008 next_2008;

    public Pasien_2511532008(String namaPasien_2008, String keluhan_2008, int nomorAntrian_2008) {
        this.namaPasien_2008 = namaPasien_2008;
        this.keluhan_2008 = keluhan_2008;
        this.nomorAntrian_2008 = nomorAntrian_2008;
        this.next_2008 = null;
    }

    public String getNamaPasien_2008() {
        return namaPasien_2008;
    }

    public void setNamaPasien_2008(String namaPasien_2008) {
        this.namaPasien_2008 = namaPasien_2008;
    }

    public String getKeluhan_2008() {
        return keluhan_2008;
    }

    public void setKeluhan_2008(String keluhan_2008) {
        this.keluhan_2008 = keluhan_2008;
    }

    public int getNomorAntrian_2008() {
        return nomorAntrian_2008;
    }

    public void setNomorAntrian_2008(int nomorAntrian_2008) {
        this.nomorAntrian_2008 = nomorAntrian_2008;
    }

    public Pasien_2511532008 getNext_2008() {
        return next_2008;
    }

    public void setNext_2008(Pasien_2511532008 next_2008) {
        this.next_2008 = next_2008;
    }
}
```

Kelas Driver

```
package pekan5_2511532008;

import java.util.*;

public class RumahSakit_2511532008 {
    private Pasien_2511532008 head_2008;
    private int counter_2008 = 1;
    private int jumlahPasien_2008 = 0;

    public void daftarkanPasien_2008(String nama, String keluhan) {
        Pasien_2511532008 pasienBaru_2008 = new Pasien_2511532008(nama, keluhan, counter_2008);
        if (head_2008 == null) {
            head_2008 = pasienBaru_2008;
        } else {
            Pasien_2511532008 temp_2008 = head_2008;
            while (temp_2008.getNext_2008() != null) {
                temp_2008 = temp_2008.getNext_2008();
            }
            temp_2008.setNext_2008(pasienBaru_2008);
        }
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_2008);
        counter_2008++;
        jumlahPasien_2008++;
    }

    public void panggilPasien_2008() {
        if (head_2008 == null) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Memanggil Pasien..");
            System.out.println("Nama: " + head_2008.getNamaPasien_2008());
            System.out.println("Nomor Antrian: " + head_2008.getNomorAntrian_2008());
            head_2008 = head_2008.getNext_2008();
            jumlahPasien_2008--;
        }
    }

    public void tampilkanAntrian_2008() {
        if (head_2008 == null) {
            System.out.println("Antrian saat ini kosong.");
        } else {
            Pasien_2511532008 current_2008 = head_2008;
            System.out.println("--- Daftar Antrian Pasien ---");
            while (current_2008 != null) {
                System.out.println("[ " + current_2008.getNomorAntrian_2008() + " ] " +
                    current_2008.getNamaPasien_2008() + " - " +
                    current_2008.getKeluhan_2008());
                current_2008 = current_2008.getNext_2008();
            }
        }
    }
}
```

```

public void cariPasien_2008(String namaCari_2008) {
    Pasien_2511532008 current_2008 = head_2008;
    boolean ditemukan_2008 = false;
    int posisi_2008 = 1;

    while (current_2008 != null) {
        if (current_2008.getNamaPasien_2008().equalsIgnoreCase(namaCari_2008)) {
            System.out.println("Pasien ditemukan!");
            System.out.println("Nama: " + current_2008.getNamaPasien_2008());
            System.out.println("Nomor Antrian: " + current_2008.getNomorAntrian_2008());
            System.out.println("Urutan dalam antrian: " + posisi_2008);
            ditemukan_2008 = true;
            break;
        }
        current_2008 = current_2008.getNext_2008();
        posisi_2008++;
    }

    if (!ditemukan_2008) {
        System.out.println("Pasien dengan nama '" + namaCari_2008 + "' tidak ditemukan.");
    }
}

public void cekStatusAntrian_2008() {
    System.out.println("Total pasien dalam antrian: " + jumlahPasien_2008);
    if (head_2008 != null) {
        System.out.println("Pasien terdepan: " + head_2008.getNamaPasien_2008() +
            " (No: " + head_2008.getNomorAntrian_2008() + ")");
    } else {
        System.out.println("Tidak ada pasien di barisan depan.");
    }
}
}

```

```

public static void main(String[] args) {
    RumahSakit_2511532008 rs_2008 = new RumahSakit_2511532008();
    Scanner input_2008 = new Scanner(System.in);
    int pilihan_2008;

    do {
        System.out.println("\n=== Antrian Rumah Sakit NIM: 2511532008 ===");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_2008 = input_2008.nextInt();
        input_2008.nextLine();

        switch (pilihan_2008) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_2008 = input_2008.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String keluhan_2008 = input_2008.nextLine();
                rs_2008.daftarkanPasien_2008(nama_2008, keluhan_2008);
                break;
            case 2:
                rs_2008.panggilPasien_2008();
                break;
            case 3:
                rs_2008.tampilkanAntrian_2008();
                break;
            case 4:
                System.out.print("Masukkan Nama Pasien yang dicari: ");
                String cari_2008 = input_2008.nextLine();
                rs_2008.cariPasien_2008(cari_2008);
                break;
            case 5:
                rs_2008.cekStatusAntrian_2008();
                break;
            case 6:
                System.out.println("Program Selesai.");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan_2008 != 6);
    input_2008.close();
}

```

OUTPUT

```
=== Antrian Rumah Sakit NIM: 2511532008 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Joni
Masukkan Keluhan: Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532008 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Asep
Masukkan Keluhan: Flu
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511532008 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Josep
Masukkan Keluhan: Demam Flu
Pasien berhasil didaftarkan! Nomor Antrian: 3

=== Antrian Rumah Sakit NIM: 2511532008 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
|--- Daftar Antrian Pasien ---
[1] Joni - Demam
[2] Asep - Flu
[3] Josep - Demam Flu
```